Keeper-Safe Near-Duplicate Groups (#9) Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Make near-duplicate groups keeper-safe: every member marked actionable is measured directly within threshold of its keeper, transitive-only members stay reviewable but can never become automatic deletion candidates, and the whole pipeline is verified against a brute-force oracle at every supported threshold (issue #9).

Architecture: Union-find components stay exactly what they are — review groups built from verified pairwise edges. What changes is the classification layered on top: a new additive `actionable` member field implements the keeper-star rule (`hamming_to_keep <= threshold`, never shadowed/hardlink/keeper), the reclaimable/duplicate accounting counts only actionable members, and transitive-only members get their own additive counters. No re-partitioning, no group-shape change, no version bump — dedup JSON stays `version: 1`, additive-only per docs/CONTRACT.md and docs/COMPATIBILITY.md.

Tech Stack: Rust, rusqlite, serde, image + existing `phash` module (`sage-dct-v1`, FROZEN), golden-fixture harness in tests/contract.rs.

Global Constraints

- **Additive-only JSON**: "a re-bless whose diff adds fields may ship; a re-bless whose diff renames or removes fields is a breaking change and must not" (docs/CONTRACT.md). `hamming_to_keep`, `keep`, `keep_reason`, `max_distance`, `reclaimable_bytes` keep their names, types, and documented meanings. Dedup stays report `version: 1` (docs/COMPATIBILITY.md).
- **`--threshold` stays capped at 0..=3** — recall guarantee of the 4-band MIH scheme; threshold semantics untouched (deferred to v1.3).
- **Downstream contract (v1.1/v1.2, already frozen in ROADMAP + future-roadmap design)**: near duplicates are never selected automatically; plans require explicit per-file selection. `actionable` is descriptive input to that future selection, never itself an action trigger.
- **Safety invariant**: BackupSage never rewrites or deletes from an archive. This change is read-only reporting.
- **`sage-dct-v1` is FROZEN** — tests may call `phash::phash`/`phash::hamming` but nothing touches src/phash.rs algorithm code.
- **Fixture regeneration**: only via `BACKUPSAGE_BLESS=1 cargo test --test contract`, diff reviewed as additive.
- **Git discipline (session-specific)**: a background autocheckpointer is the sole git writer during implementation — tasks do NOT run `git add`/`git commit`; WIP lands

automatically each minute. The final reviewed commit happens once, after the checkpointer is stopped.
- Gate for every task: `cargo test` green, then at the end `cargo clippy --all-targets -- -D warnings` and `cargo fmt --check` clean.

Files

- Modify: `src/dedup.rs` — classification helper, `near_components` extraction, accounting, deterministic sort, oracle unit tests
- Modify: `src/report.rs` — additive fields on `Member`, `Group`, `Summary`, `ReportParams`
- Modify: `src/main.rs` — text renderer additions (`render_dedup_report`)
- Modify: `tests/common/mod.rs` — deterministic 3-image near-chain helper
- Modify: `tests/dedup.rs` — chain integration test, JSON field-pin update
- Modify: `tests/contract.rs` — chain corpus, new fixture, orphan list 7→8
- Create: `tests/fixtures/contract/dedup_chain.json` (blessed)
- Re-bless: `tests/fixtures/contract/dedup.json`, `tests/fixtures/contract/dedup_skips.json` (additive field diff only)
- Modify: `docs/CONTRACT.md`, `docs/COMPATIBILITY.md`, `docs/ROADMAP.md`
- Create: `docs/adr/0004-keeper-star-actionable-groups.md`; modify `docs/adr/README.md`

Task 1: `member_is_actionable` — the keeper-star rule as a pure function

Files: - Modify: `src/dedup.rs` (helper near `pick_keep`, tests in existing `#[cfg(test)] mod tests`)

Interfaces: - Produces: `fn member_is_actionable(kind: &str, is_keep: bool, shadowed: bool, hardlink: bool, hamming_to_keep: Option<u32>, threshold: u32) -> bool` — consumed by Task 4 (report assembly) and Task 3 (oracle).

- [ ] **Step 1: Write the failing tests** (append inside `mod tests` in src/dedup.rs)

```
#[test]
fn actionable_is_keeper_star_only() {
    use super::member_is_actionable as act;
    // near: directly measured within threshold → actionable
    assert!(act("near", false, false, false, Some(0), 3));
    assert!(act("near", false, false, false, Some(3), 3));
    // near: transitive-only (beyond threshold) → review-only
    assert!(!act("near", false, false, false, Some(4), 3));
    // threshold is the run's threshold, not MAX: distance 2 at threshold 1 is NOT
safe
    assert!(!act("near", false, false, false, Some(2), 1));
    // missing distance can never be actionable
    assert!(!act("near", false, false, false, None, 3));
    // exact groups are byte-identical — always actionable when eligible
    assert!(act("exact", false, false, false, Some(0), 3));
    assert!(act("exact", false, false, false, None, 0));
    // keeper, shadowed and hardlink members are never actionable
    assert!(!act("near", true, false, false, Some(0), 3));
    assert!(!act("exact", false, true, false, Some(0), 3));
    assert!(!act("exact", false, false, true, Some(0), 3));
    // unknown kind fails closed
    assert!(!act("weird", false, false, false, Some(0), 3));
}
```

- [ ] **Step 2: Run to verify failure**

Run: `cargo test --lib actionable_is_keeper_star_only` Expected: compile FAIL — `member_is_actionable` not found.

- [ ] **Step 3: Implement** (place after `pick_keep` in src/dedup.rs)

```
/// The keeper-star rule (issue #9): a member is an *actionable* duplicate
/// candidate only when its distance to the keeper was directly measured and
/// is within the run's threshold. Members that are in the group only through
/// a chain of pairwise matches (transitive-only) stay reviewable but must
/// never be selected automatically. Keepers, shadowed rows and hardlinks are
/// never actionable. Fails closed on anything unknown.
fn member_is_actionable(
    kind: &str,
    is_keep: bool,
    shadowed: bool,
    hardlink: bool,
    hamming_to_keep: Option<u32>,
    threshold: u32,
) -> bool {
    if is_keep || shadowed || hardlink {
        return false;
    }
    match kind {
        // Exact members share a content hash with the keeper — identity, not
        // an estimate; distance is 0 by definition.
        "exact" => true,
        "near" => matches!(hamming_to_keep, Some(d) if d <= threshold),
        _ => false,
    }
}
```

- [ ] **Step 4: Run to verify pass**

Run: `cargo test --lib actionable_is_keeper_star_only` Expected: PASS. (No commit step — autocheckpointer owns git.)

Task 2: Extract `near_components` (pure refactor, no behavior change)

Files: - Modify: `src/dedup.rs:176-218` (near-group construction in `run_dedup`)

Interfaces: - Produces: `fn near_components(hashes: &[u64], threshold: u32, cap: usize, skipped: &mut u64) -> Vec<Vec<usize>>` — sorted components (each sorted, size ≥ 2) over local indices. Consumed by Task 3's oracle and by `run_dedup`.

- [ ] **Step 1: Implement the extraction** (replace the `mih_pairs` + union-find + clusters block inside `run_dedup`, and add the new function near `mih_pairs`)

```
/// Connected components (size ≥ 2) of the verified within-threshold pair
/// graph — the *review groups*. Component membership only proves a chain of
/// pairwise matches; it never implies every member is within threshold of
/// every other. Keeper-star classification happens later, per member.
fn near_components(
    hashes: &[u64],
    threshold: u32,
    cap: usize,
    skipped: &mut u64,
) -> Vec<Vec<usize>> {
    let pairs = mih_pairs(hashes, threshold, cap, skipped);
    let mut uf = UnionFind::new(hashes.len());
    for &(a, b) in &pairs {
        uf.union(a, b);
    }
    let mut clusters: HashMap<usize, Vec<usize>> = HashMap::new();
    for i in 0..hashes.len() {
        clusters.entry(uf.find(i)).or_default().push(i);
    }
    let mut components: Vec<Vec<usize>> = clusters
        .into_values()
        .filter(|c| c.len() > 1)
        .collect();
    for c in &mut components {
        c.sort_unstable();
    }
    components.sort();
    components
}
```

In `run_dedup`, the near-group block becomes:

```
        let local_hashes: Vec<u64> = image_idxs.iter().map(|&i|
rows[i].phash.unwrap() as u64).collect();
        for members in near_components(&local_hashes, p.threshold, p.bucket_cap,
&mut near_buckets_skipped) {
            let globals: Vec<usize> = members.iter().map(|&l|
image_idxs[l]).collect();
            for &m in &globals {
                in_near_group[m] = true;
            }
            groups_raw.push(("near".into(), globals, 0));
        }
```

- [ ] **Step 2: Verify no behavior change**

Run: `cargo test` Expected: all existing tests PASS (including `mih_equals_brute_force_on_random_corpus`, untouched).

Task 3: Grouping-level brute-force oracle at every threshold (AC1)

Files: - Modify: `src/dedup.rs` (`mod tests`)

Interfaces: - Consumes: `near_components` (Task 2), `member_is_actionable` (Task 1), `phash::hamming`, `UnionFind`.

- [ ] **Step 1: Write the failing/verifying oracle test**

```
/// Issue #9 AC1: the full grouping pipeline (MIH candidates → union-find →
/// components) must match a brute-force pair oracle at every supported
/// threshold, and keeper-star classification must admit exactly the members
/// whose directly measured distance to the keeper is within threshold.
#[test]
fn grouping_matches_brute_force_oracle_at_every_threshold() {
    // Deterministic corpus: random hashes plus planted CHAINS so the
    // transitive-only case genuinely occurs (anti-vacuity, asserted below).
    let mut state = 0x9E37_79B9_7F4A_7C15u64;
    let mut rand = || {
        state ^= state << 13;
        state ^= state >> 7;
        state ^= state << 17;
        state
    };
    let mut hashes: Vec<u64> = (0..1200).map(|_| rand()).collect();
    // Planted chains A–B–C: d(A,B)=3, d(B,C)=3, disjoint bit sets so
    // d(A,C)=6 — inside one component at threshold 3, far beyond it pairwise.
    for i in 0..40 {
        let a = hashes[i * 13];
        let b = a ^ (0b111u64 << ((i * 5) % 60));
        let c = b ^ (0b111u64 << (((i * 5) + 7) % 60));
        hashes.push(b);
        hashes.push(c);
    }

    for threshold in 0..=3u32 {
        let mut skipped = 0u64;
        let components = near_components(&hashes, threshold, DEFAULT_BUCKET_CAP,
&mut skipped);
        assert_eq!(skipped, 0, "oracle corpus must not hit the bucket cap");

        // Brute-force oracle: union every pair within threshold, then
        // compare the resulting components with the pipeline's.
        let mut uf = UnionFind::new(hashes.len());
        for a in 0..hashes.len() {
            for b in (a + 1)..hashes.len() {
                if phash::hamming(hashes[a], hashes[b]) <= threshold {
                    uf.union(a, b);
                }
            }
        }
        let mut oracle_clusters: HashMap<usize, Vec<usize>> = HashMap::new();
        for i in 0..hashes.len() {
            oracle_clusters.entry(uf.find(i)).or_default().push(i);
        }
        let mut oracle: Vec<Vec<usize>> = oracle_clusters
            .into_values()
            .filter(|c| c.len() > 1)
            .collect();
        for c in &mut oracle {
            c.sort_unstable();
```

```
        }
        oracle.sort();
        assert_eq!(components, oracle, "component mismatch at threshold
{threshold}");

        // Keeper-star: for EVERY possible keeper choice in every component,
        // classification admits exactly the members measured within threshold.
        let mut transitive_only_seen = false;
        for comp in &components {
            for &keeper in comp {
                for &m in comp {
                    let d = phash::hamming(hashes[keeper], hashes[m]);
                    let actionable =
                        member_is_actionable("near", m == keeper, false, false,
Some(d), threshold);
                    assert_eq!(
                        actionable,
                        m != keeper && d <= threshold,
                        "keeper-star violated: keeper {keeper}, member {m}, d {d},
t {threshold}"
                    );
                    if m != keeper && d > threshold {
                        transitive_only_seen = true;
                    }
                }
            }
        }
        // Anti-vacuity: at threshold 3 the planted chains MUST produce
        // transitive-only members, or this test is testing nothing.
        if threshold == 3 {
            assert!(transitive_only_seen, "corpus produced no transitive-only
member");
        }
    }
}
```

- [ ] **Step 2: Run it**

Run: `cargo test --lib grouping_matches_brute_force_oracle_at_every_threshold`
Expected: PASS (components already match by construction; the keeper-star and anti-vacuity assertions are the new evidence). If the component comparison fails, that is a real found bug — stop and diagnose before proceeding.

Task 4: Additive report fields + honest accounting

Files: - Modify: `src/report.rs` (`Member`, `Group`, `Summary`, `ReportParams`) - Modify: `src/dedup.rs` (`run_dedup` assembly loop, params echo) - Modify: `tests/dedup.rs` (`json_contract_field_names_are_stable`, near-pair test)

Interfaces: - Produces JSON fields (all additive, consumed by Tasks 6-8): - `Member.actionable: bool` - `Group.review_only_bytes: u64` - `Summary.transitive_only_files:`

`usize`, `Summary.review_only_bytes: u64` - `ReportParams.actionable_rule: String` = `"near:direct-to-keeper-within-threshold; exact:always; shadowed/hardlink:never"`

- [ ] **Step 1: Extend the failing field-pin test** (tests/dedup.rs `json_contract_field_names_are_stable`): add to the member-key assertions `"actionable"`, to the group-key assertions `"review_only_bytes"`, to the summary-key assertions `"transitive_only_files"` and `"review_only_bytes"`, and to the params-key assertions `"actionable_rule"`. Also extend `near_duplicate_images_group_perceptually`: the dup member of the 2-image pair must have `"actionable": true` (its distance 2 ≤ threshold 3) and the keeper `"actionable": false`.
- [ ] **Step 2: Run to verify failure**

Run: `cargo test --test dedup` Expected: FAIL — missing keys.

- [ ] **Step 3: Implement.** In `src/report.rs` add (with doc comments in the existing style):

```
    // Member — after `keep_reason`:
    /// Keeper-star safety (issue #9): true only when this member's distance
    /// to the keeper was directly measured and is within the run's threshold
    /// (exact members: identity). Transitive-only members are false and must
    /// never be auto-selected by future action planning.
    pub actionable: bool,
```

```
    // Group — after `reclaimable_bytes`:
    /// Bytes held by transitive-only members — reviewable, not reclaimable.
    pub review_only_bytes: u64,
```

```
    // Summary — after `reclaimable_bytes`:
    /// Near-dup members beyond the threshold relative to their keeper.
    pub transitive_only_files: usize,
    /// Bytes held by transitive-only members across all groups.
    pub review_only_bytes: u64,
```

```
    // ReportParams — after `keep_policy`:
    /// How `Member.actionable` is derived (issue #9, keeper-star rule).
    pub actionable_rule: String,
```

In `src/dedup.rs` assembly loop (currently `if !is_keep { if r.shadowed() { … } else if !r.is_hardlink() { reclaimable += r.size; duplicate_files += 1; } }`):

```
            let actionable = member_is_actionable(
                &kind,
                is_keep,
                r.shadowed(),
                r.is_hardlink(),
                hamming,
                p.threshold,
            );
            if !is_keep {
                if r.shadowed() {
                    shadowed_bytes += r.size;
                } else if !r.is_hardlink() {
                    if actionable {
                        reclaimable += r.size;
                        duplicate_files += 1;
                    } else {
                        review_only += r.size;
                        transitive_only_files += 1;
                    }
                }
            }
```

with `let mut review_only = 0u64;` per group, `let mut transitive_only_files = 0usize;` and `let mut review_only_total = 0u64;` alongside the existing report-wide accumulators; `actionable` goes into the `Member { … }` literal; `review_only_bytes: review_only` into `Group { … }`; the two totals into `Summary { … }`; and in the `ReportParams { … }` literal:

```
            actionable_rule: "near:direct-to-keeper-within-threshold; exact:always;
\
                              shadowed/hardlink:never"
                .into(),
```

`keep_policy` stays byte-identical.

- [ ] **Step 4: Run the suite**

Run: `cargo test` Expected: tests/dedup.rs and unit tests PASS; **tests/contract.rs FAILS** (fixtures lack the new fields) — expected and left red until Task 8's deliberate re-bless. Everything else green.

Task 5: Deterministic group ordering (kills the documented HashMap-order fragility)

Files: - Modify: `src/dedup.rs:340-352` (sort arms)

- [ ] **Step 1: Write the failing test** (tests in src/dedup.rs are hash-level; this is simplest as an integration assertion — append to Task 6's chain test instead if preferred; otherwise pin at unit level that `near_components` output is sorted, which Task 2 already guarantees. The load-bearing change is the tie-break below.)

- [ ] **Step 2: Implement stable tie-breaks** — replace the three `sort_by_key` arms so equal primary keys fall back to the first member (groups are already internally sorted keeper-first, then by archive/file id):

```
    let first_key = |g: &Group| {
        g.members
            .iter()
            .map(|m| (m.archive_id, m.file_id))
            .min()
            .unwrap_or((i64::MAX, i64::MAX))
    };
    match p.sort {
        SortKey::Wasted => {
            groups.sort_by_key(|g| (std::cmp::Reverse(g.reclaimable_bytes),
first_key(g)))
        }
        SortKey::Count => {
            groups.sort_by_key(|g| (std::cmp::Reverse(g.members.len()),
first_key(g)))
        }
        SortKey::Newest => groups.sort_by_key(|g| {
            (
                std::cmp::Reverse(
                    g.members
                        .iter()
                        .find(|m| m.keep)
                        .and_then(|m| m.best_ts_unix)
                        .unwrap_or(i64::MIN),
                ),
                first_key(g),
            )
        }),
    }
```

- [ ] **Step 3: Run the suite**

Run: `cargo test` Expected: same pass/fail set as after Task 4 (contract still red, all else green — fixture group order must not change, since current fixtures have distinct reclaimable_bytes).

Task 6: Real-image chain — helper + end-to-end integration test

Files: - Modify: `tests/common/mod.rs` - Modify: `tests/dedup.rs`

Interfaces: - Produces: `pub fn near_chain_pngs() -> (Vec<u8>, Vec<u8>, Vec<u8>)` — PNGs (a, b, c) where, under `phash::phash`: keeper-forcing `a` is 640×480 (highest pixel count), `d(a,b) ∈ 1..=3`, `d(b,c) ∈ 1..=3`, `d(a,c) > 3`. Found by bounded deterministic search; panics with a clear message if the space yields nothing (loud fixture failure, never silent).

- [ ] **Step 1: Implement the helper** (tests/common/mod.rs; `image` and the backupsage lib are already dev-dependencies of the test crate)

```
/// Deterministic 3-image near-dup chain for keeper-safety tests (issue #9):
/// returns (a, b, c) PNGs where a is 640×480 (forced keeper by resolution),
/// hamming(pa, pb) and hamming(pb, pc) are within 3, and hamming(pa, pc) > 3
/// — so c lands in the group only transitively. Search is bounded and
/// deterministic; a miss is a loud panic, never a silent skip.
pub fn near_chain_pngs() -> (Vec<u8>, Vec<u8>, Vec<u8>) {
    use backupsage::phash;
    let hash_of = |png: &[u8]| {
        phash::phash(&image::load_from_memory(png).expect("test png decodes"))
    };
    for seed in 0..64u64 {
        let base = png_bytes(seed, 320, 240);
        // Upscaled twin: same picture, highest resolution → forced keeper.
        let big = {
            let img = image::load_from_memory(&base).expect("test png decodes");
            let up = img.resize_exact(640, 480,
image::imageops::FilterType::Triangle);
            let mut out = Vec::new();
            up.write_to(&mut std::io::Cursor::new(&mut out),
image::ImageFormat::Png)
                .expect("png encodes");
            out
        };
        let (pa, pbase) = (hash_of(&big), hash_of(&base));
        if phash::hamming(pa, pbase) > 1 {
            continue; // upscale drifted too far to leave room for b
        }
        for delta_b in [24u8, 32, 40, 48, 56, 64, 80, 96] {
            let b = png_bytes_brightened(seed, 320, 240, delta_b);
            let pb = hash_of(&b);
            if !(1..=3).contains(&phash::hamming(pa, pb)) {
                continue;
            }
            for delta_c in [96u8, 112, 128, 144, 160, 176, 192, 208] {
                if delta_c <= delta_b {
                    continue;
                }
                let c = png_bytes_brightened(seed, 320, 240, delta_c);
                let pc = hash_of(&c);
                if (1..=3).contains(&phash::hamming(pb, pc)) && phash::hamming(pa,
pc) > 3 {
                    return (big, b, c);
                }
            }
        }
    }
    panic!(
        "near_chain_pngs: bounded search found no A-B-C chain — \
         phash behavior changed; re-tune the search space deliberately"
    );
}
```

- [ ] **Step 2: Write the failing end-to-end test** (tests/dedup.rs; follow the file's existing archive-building pattern — same helpers `near_duplicate_images_group_perceptually` uses to build tars, index them, and run dedup with `--json`)

```
/// Issue #9 end-to-end: a transitive-only chain member is reported,
/// reviewable, and excluded from every actionable count.
#[test]
fn transitive_chain_member_is_review_only_not_actionable() {
    let (a_png, b_png, c_png) = common::near_chain_pngs();
    // Two archives so the group is cross-archive like real corpora:
    // alpha: photos/orig.png (a — keeper by resolution), photos/bright.png (b)
    // beta:  export/faded.png (c — transitive-only)
    // [build tars, index both, master-sync, run dedup --json exactly as the
    //  existing near_duplicate_images_group_perceptually test does]
    let v: serde_json::Value = /* parsed dedup --json output */;

    let groups = v["groups"].as_array().unwrap();
    assert_eq!(groups.len(), 1, "chain must form ONE review group");
    let g = &groups[0];
    assert_eq!(g["match_kind"], "near");
    let members = g["members"].as_array().unwrap();
    assert_eq!(members.len(), 3);

    let by_path: std::collections::HashMap<&str, &serde_json::Value> = members
        .iter()
        .map(|m| (m["path"].as_str().unwrap(), m))
        .collect();
    let keeper = by_path["photos/orig.png"];
    let direct = by_path["photos/bright.png"];
    let transitive = by_path["export/faded.png"];

    assert_eq!(keeper["keep"], true);
    assert_eq!(keeper["actionable"], false);
    assert_eq!(direct["keep"], false);
    assert_eq!(direct["actionable"], true);
    assert!(direct["hamming_to_keep"].as_u64().unwrap() <= 3);
    assert_eq!(transitive["keep"], false);
    assert_eq!(transitive["actionable"], false);
    assert!(transitive["hamming_to_keep"].as_u64().unwrap() > 3);

    // Honest accounting: the transitive member's bytes are review-only.
    let c_size = transitive["size"].as_u64().unwrap();
    assert_eq!(g["review_only_bytes"].as_u64().unwrap(), c_size);
    assert_eq!(
        g["reclaimable_bytes"].as_u64().unwrap(),
        direct["size"].as_u64().unwrap()
    );
    let s = &v["summary"];
    assert_eq!(s["transitive_only_files"].as_u64().unwrap(), 1);
    assert_eq!(s["review_only_bytes"].as_u64().unwrap(), c_size);
    assert_eq!(s["duplicate_files"].as_u64().unwrap(), 1);
}
```

- [ ] **Step 3: Run to verify failure, then wire the corpus** (the bracketed build section) using the existing tar/index/sync pattern from `near_duplicate_images_group_perceptually`; run again.

Run: `cargo test --test dedup transitive_chain_member_is_review_only_not_actionable` Expected: PASS once wired. If the group unexpectedly splits or the keeper is not `orig.png`, the helper's chain properties vs `pick_keep`'s resolution-first rule are out of sync — diagnose, don't loosen assertions.

Task 7: Text renderer — review-only visible to humans

Files: - Modify: `src/main.rs` (`render_dedup_report`, member row + summary footer) - Modify: `tests/cli.rs` (dedup text assertions, following existing patterns)

- [ ] **Step 1: Failing CLI test**: extend the existing dedup text-output test in tests/cli.rs (or add one following its pattern) building the Task 6 chain corpus and asserting the text output contains `[review-only]` on the transitive member's row and a summary line matching `1 near-duplicate member beyond threshold — review manually, never auto-deletable`.
- [ ] **Step 2: Implement.** In the member-row extras (where `[dist N]`, `[shadowed]`, `[sparse]`, `[hardlink]` are emitted): after the `[dist N]` block add

```
                if !m.keep && !m.shadowed && m.hardlink_of.is_none() && !
m.actionable {
                    extras.push("[review-only]".to_string());
                }
```

(adapt to the actual accumulation style at src/main.rs:496-575 — extras there are pushed as formatted strings). In the summary footer, after the `near_buckets_skipped` conditional:

```
    if report.summary.transitive_only_files > 0 {
        println!(
            "{} near-duplicate member{} beyond threshold — review manually, never
auto-deletable ({})",
            report.summary.transitive_only_files,
            if report.summary.transitive_only_files == 1 { "" } else { "s" },
            human_bytes(report.summary.review_only_bytes),
        );
    }
```

(use the file's existing byte-formatting helper — check its actual name at src/main.rs: 576-623 and match it).

- [ ] **Step 3: Run**

Run: `cargo test --test cli` Expected: PASS.

Task 8: Contract work — chain fixture, re-bless, docs

Files: - Modify: `tests/contract.rs` (corpus + orphan list) - Create: `tests/fixtures/contract/dedup_chain.json` - Re-bless: `tests/fixtures/contract/dedup.json`, `dedup_skips.json` - Modify: `docs/CONTRACT.md`, `docs/COMPATIBILITY.md`

- [ ] **Step 1: Add the chain scenario to tests/contract.rs**: a new test `dedup_chain_json_matches_fixture` that builds a dedicated mini-corpus (two archives, the three `near_chain_pngs()` images laid out as in Task 6), runs `dedup --json --threshold 3`, and `assert_matches_fixture("dedup_chain.json", …)`. Add `"dedup_chain.json"` to the hardcoded fixture-name list (7→8) in the orphan guard.
- [ ] **Step 2: Re-bless deliberately**

Run: `BACKUPSAGE_BLESS=1 cargo test --test contract` then `cargo test --test contract` Expected: second run PASS. Then `git diff --stat` the fixtures (the checkpointer will have committed the pre-bless state, so the diff is inspectable): `dedup.json`/`dedup_skips.json` diffs must show ONLY added keys (`actionable`, `review_only_bytes`, `transitive_only_files`, `actionable_rule`) with values consistent with the 2-member pair (both existing members: `actionable` true for the dup, false for keeper; `review_only_bytes: 0`; `transitive_only_files: 0`). `dedup_chain.json` is new and must pin `actionable: false` with `hamming_to_keep > 3` on the transitive member — the non-degenerate value ADR 0003's residual-gaps note demands. Any removed/renamed key = stop, fix, re-bless.

- [ ] **Step 3: Update docs.**
- `docs/CONTRACT.md`: fixture count 7→8 with `dedup_chain.json` listed; note under the dedup surface: "v1.0.2 (#9) added `actionable`, `review_only_bytes`, `transitive_only_files`, `actionable_rule` — additive, still version 1."
- `docs/COMPATIBILITY.md`: in the JSON row, record the additive v1.0.2 field additions and that `reclaimable_bytes`/`duplicate_files` now count only keeper-star-safe members (correctness fix: previously inflated by transitive-only members whose safety was never measured).

- [ ] **Step 4: Full gate**

Run: `cargo test && cargo clippy --all-targets -- -D warnings && cargo fmt --check` Expected: all green, all targets.

Task 9: ADR 0004 + roadmap tick

Files: - Create: `docs/adr/0004-keeper-star-actionable-groups.md` (read 0003 first; match its Context/Decision/Alternatives/Consequences structure and index style exactly) - Modify: `docs/adr/README.md` (index entry) - Modify: `docs/ROADMAP.md` (tick `- [x] Make near-duplicate groups keeper-safe` with `(#9)` once merged, matching the v1.0.1 section's style)

ADR content requirements: Context = union-find transitivity vs the v1.2 "near duplicates are never selected automatically" contract; Decision = keeper-star classification (additive `actionable`, honest accounting), review groups unchanged; Alternatives = (a) explicit pairwise edge lists in JSON — rejected: O(n^2) payload, overspecifies what v1.1 planning needs, and the safety question is only ever "safe relative to the keeper you retain"; (b) re-

partitioning clusters so every group is pairwise-within-threshold — rejected: destroys the human review affordance, unstable under member insertion, and still needs a keeper rule; Consequences = fixtures re-blessed additively, `reclaimable_bytes` semantics now honest (bug-fix value change, documented), v1.1 plan generation consumes `actionable` as its hard selection floor.

Task 10: Adversarial review, final commit, PR

- [ ] **Step 1:** Run the adversarial review fleet (independent reviewers: correctness/oracle-vacuity, contract-compatibility, test-quality; session-model verify pass on majors). Fix everything confirmed.
- [ ] **Step 2:** Full gate again: `cargo test && cargo clippy --all-targets -- -D warnings && cargo fmt --check`.
- [ ] **Step 3:** Stop the autocheckpointer (remove its marker file), then make the final commit as `claude_2010` and push:

```
git -c user.name=claude_2010 -c
user.email=262510778+tom2025b@users.noreply.github.com \
  commit -m "feat: keeper-safe near-duplicate groups (#9)" # full message per repo
style
git push -u origin issue-9-keeper-safe-groups
```

- [ ] **Step 4:** Open the PR (body: what/why, oracle evidence, fixture-diff summary, gate results), merge to main after checks, **never delete the branch**, close #9 with evidence, tick ROADMAP (in the PR itself), update worklog + journal.

Self-Review

- **Spec coverage:** AC1 → Tasks 3 (hash-level, thresholds 0..=3) + 6 (real-image e2e); AC2 → Tasks 1/4 (`actionable` = measured-direct-≤-threshold only); AC3 → Tasks 4/6/7 (review-only members reported, tagged, counted, excluded from reclaimable); AC4 → representation only (plans are v1.1): `actionable` is descriptive, `actionable_rule` documents it, ADR 0004 records the v1.2 selection floor. Scope line "avoid implying unmeasured pairwise guarantees" → Task 2 doc comment + ADR; "separate review components from actionable candidates" → the whole design.
- **Placeholder scan:** Task 6 Step 2 has one deliberate bracketed section (corpus wiring) pointing at a concrete existing test to mirror — acceptable because the exact tar-building helper signatures live in that file and must be copied from it, not invented here. Task 7 names exact insertion points but defers to the file's real accumulation style with line ranges given.
- **Type consistency:** `member_is_actionable` signature identical in Tasks 1/3/4; `near_components` identical in Tasks 2/3; JSON field names identical across Tasks 4/6/7/8.

Signed: thomas2025 · 2026-08-02T00:00:00-04:00 (timestamp set at write time by session; see git log for precise time)